

8. ALMACENAMIENTO DE OBJETOS EN FICHEROS. PERSISTENCIA.	
SERIALIZACIÓN.....	1
8.1. Serialización.....	1
8.2. Deserialización.....	2

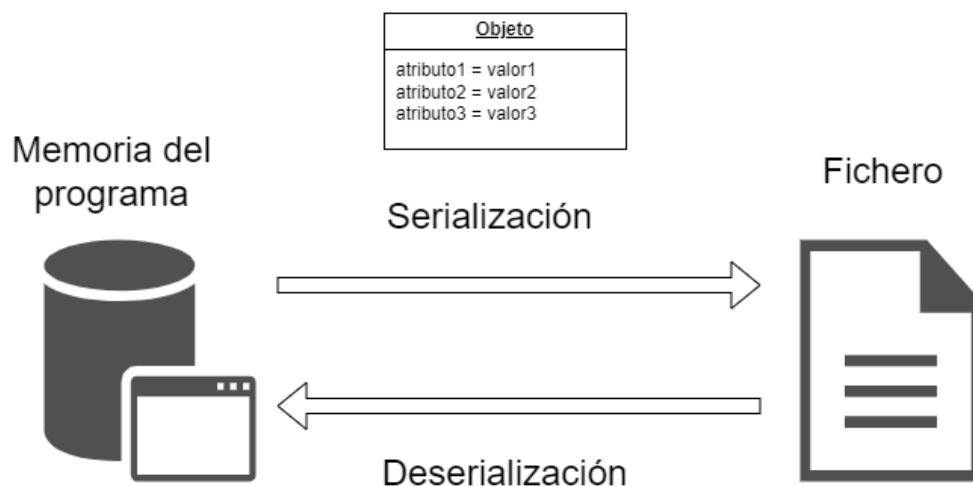
8. ALMACENAMIENTO DE OBJETOS EN FICHEROS.

PERSISTENCIA. SERIALIZACIÓN.

Además de datos binarios, o texto, java nos permite persistir, es decir, almacenar y recuperar, objetos en ficheros.

Esto consistirá en coger objetos que tengamos almacenados en memoria y los podamos introducir en los ficheros en formato binario, de tal forma que cuando recuperemos esos datos podremos guardarlos de nuevo en objetos del mismo tipo en memoria.

A la operación de almacenar objetos en ficheros le damos el nombre de serialización, mientras que la operación inversa, es decir, recoger los datos del fichero y guardarlo en un objeto le denominamos deserialización.



8.1. Serialización

A la hora de almacenar objetos en ficheros vamos a hacer uso de las clases `java.io.FileOutputStream` y `java.io.ObjectOutputStream`. La primera será la que realice la escritura en el fichero, mientras que la segunda se encargará de transformar el objeto a una serie de bytes que se puedan escribir en el fichero.

En primer lugar crearemos un objeto `FileOutputStream`, y luego pasaremos dicho objeto al constructor de `ObjectOutputStream`.

```
protected ObjectOutputStream() throws IOException, SecurityException
```

Después almacenaremos el objeto a través del método `writeObject`, pasando como parámetro el objeto en cuestión.

```
Objeto objeto = new Objeto("Nombre1", 1, "Descripción1");
FileOutputStream fos = new FileOutputStream("objetos.dat");
ObjectOutputStream oos = new ObjectOutputStream(fos);

oos.writeObject(objeto);
oos.close();
fos.close();
```

Es importante tener en cuenta que solo podremos serializar objetos de clases que implementen la interfaz `Serializable`. En el caso de nuestras propias clases, debemos hacer que nuestra clase implemente dicha interfaz.

```
public class Objeto implements Serializable {
    private String nombre;
    private Integer id;
    private String descripcion;

    ...
}
```

8.2. Deserialización

Para recuperar los objetos de un fichero utilizaremos las clases `java.io.FileInputStream` y `ObjectInputStream`.

En esta ocasión crearemos un objeto `FileInputStream` y luego se lo pasaremos como atributo al constructor de `ObjectInputStream`.

```
public ObjectInputStream(InputStream in) throws IOException
```

Para extraer los objetos haremos uso del método `readObject`, el cual tendremos que utilizar con un casteo explícito para almacenar la información en un objeto de su clase.

```
Objeto objeto2;
FileInputStream fis = new FileInputStream("objetos.dat");
ObjectInputStream ois = new ObjectInputStream(fis);
```

```
objeto2 = (Objeto) ois.readObject();
ois.close();
System.out.println(objeto2.toString());
```

En el caso de necesitar leer todos los objetos de un fichero, debemos comprobar que en el `FileInputStream` aún quedan caracteres por leer, antes de realizar cada lectura.

Veamos un ejemplo:

```
FileInputStream fis = null;
ObjectInputStream ois = null;
Persona persona;
try {
    fis = new FileInputStream(ruta);
    ois = new ObjectInputStream(fis);

    do{
        if(fis.available()==0)
            break;

        persona =(Persona) ois.readObject();
        System.out.println(persona.toString());
    } while (true);
} catch (Exception e) {
    e.toString();
}
```

Links a la ayuda de java:

<https://docs.oracle.com/javase/8/docs/api/java/io/ObjectOutputStream.html>

<https://docs.oracle.com/javase/8/docs/api/java/io/ObjectInputStream.html>

Ahora trabajemos con la serialización y la deserialización.

*Let's
work!*